

Library Management System (LMS)

SRS Document

1. **Introduction:** - The Library Management System (LMS) is a web-based application designed to manage library activities digitally. It allows users to search and filter books, view book details and availability, request books, issue or purchase e-books and physical books where applicable, make online payments, manage their profiles, view invoices, and track their order, browsing, and reading history.

The system also provides an Admin module through which administrators can manage books and inventory, add, update, or remove books, review and fulfil user requests, manage e-books, and monitor user activities.

The main goal of the LMS is to reduce manual work, centralize library operations, and provide a simple and user-friendly platform for managing both physical and digital books.

2. **Purpose:** - The main purpose of the Library Management System is to:
 - Digitize the process of searching, issuing, purchasing, and managing books.
 - Allow users to easily search and filter available books.
 - Provide users with an online facility to request and access e-books for a fixed period.
 - Manage library inventory and keep book availability up to date.
 - Provide secure online payment functionality whenever payment is required.
 - Maintain records of user orders, issued books, e-book reading activity, payments, and transactions.
 - Allow users to track their orders and access details related to their e-book validity period.
 - Provide an issue/order ticket containing relevant details such as the issue or access period. The ticket can be delivered to the user's registered email address and at user's registered WhatsApp Number.
 - Allow administrators to manage books, users, inventory, and requests, and to create additional administrator accounts with limited or full permissions to distribute administrative work.
 - Provide personalized book recommendations based on a user's previous reading or activity history.
 - Allow users to review books and share their experience with other users.
3. **Users:** - The system will primarily have two types of users: User/Issuer and Administrator.

3.1. Issuer: - An Issuer will be able to:

- Register and log in to the system.
- Search and filter books based on available criteria.
- View book details and current availability.
- Access or issue e-books and purchase physical books where applicable.
- Request a physical book that is not currently available and ask the administrator to make it available for purchase.
- Make online payments.
- View invoices and payment details.
- Manage personal profile information.
- View order and browsing history.
- View e-book access and reading history.
- Receive book recommendations based on previous activity and reading history.
- Save books to a Wishlist.
- Receive notifications and reminders for books saved to the Wishlist.
- Receive notifications about available offers or discounts, if applicable.
- Submit reviews and ratings for books.

3.2 Admin: - An Administrator will be able to:

- Log in and log out securely.
- Manage the overall library system.
- Add, update, deactivate, or delete books.
- Manage and monitor book inventory.
- View and manage user requests and fulfil them based on book availability.
- Approve or reject book issue or access requests.
- Manage and add e-books based on availability, popularity, or other library criteria.
- Manage users and monitor relevant user activities.
- Create additional administrator accounts.
- Assign tasks and permissions to other administrators, with either limited or full access depending on the administrator's role.

4. **Technical Specification:** - LMS can be implemented as Web-based Client-Server application

4.1 Frontend: - The frontend will provide an interactive user interface that is simple, clear, and easy to use for both users and administrators.

Possible technologies that can be used are: -

1. HTML
2. CSS
3. JavaScript
4. React

4.2 Backend: - The backend will handle authentication, business logic, book and inventory operations, user and role management, issue and return operations, communication with external APIs and the database, payment-related processing, and notification management.

Possible Technologies that can be used are: -

1. Java- Spring Boot
2. Python- Fast API
3. Node.js- Express

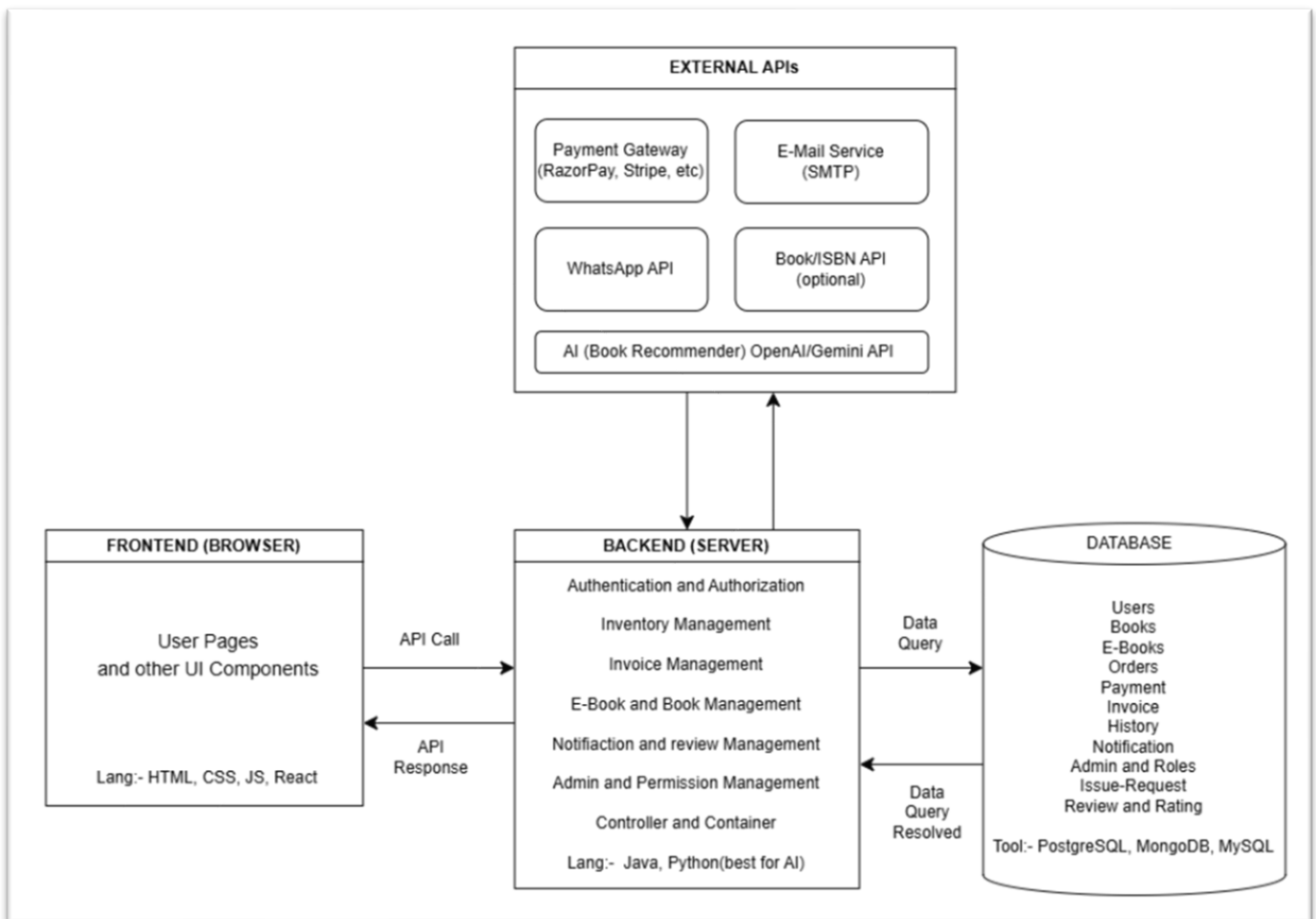
4.3 Database: - The database will store user information, book information, inventory records, issue records, orders, payments, invoices, reading history, reviews, Wishlist information, and notifications.

Possible Technologies that can be used are: -

1. MySQL
2. PostgreSQL
3. MongoDB

4.4 Architecture: - The system will follow a client-server architecture. Users and administrators will interact with the frontend, which communicates with the backend through REST APIs. The backend will communicate with the database and, where required, with external services such as payment, email, WhatsApp, and book information APIs.

Basic Flow: -



5. **Functional Requirements:** - It describes the features and operations that the system is expected to provide.

5.1 User Registration and Login

- The system shall provide a registration/signup facility.
- The system shall allow registered users to log in securely.
- The system shall validate user credentials before granting access.
- The system shall provide logout functionality.
- The system may provide a password reset facility.

5.2 Book Search and Filtering

- Users shall be able to search for books.
- Users shall be able to search by book title, author, category, or ISBN.
- Users shall be able to filter books based on available categories and other supported attributes.
- The system shall display the current availability of books.

5.3 Book Details

The system shall display information such as:

- Book title
- Author
- ISBN

- Category
- Description
- Publication information
- Availability status
- Number of copies available
- E-book availability, where applicable
- User ratings and reviews, where available

5.4 Book Issue / Request

- Users shall be able to request a book.
- The system shall check book availability before accepting the request.
- An issue request shall be sent to the admin for approval.
- Admin shall be able to approve or reject the request.
- The system shall record the issue date and expected return/due date.
- The system shall update book availability after successful issue.
- The system shall generate an issue/access ticket containing relevant details.

5.5 E-Book Management

- Users shall be able to view available e-books.
- Users shall be able to purchase or access e-books where applicable.
- The system shall maintain e-book access and reading history.
- The system may recommend e-books based on previous reading history.
- The system shall provide the expiry or validity period in the user's issue/access ticket.

5.6 Book Purchase / Order

- Users shall be able to place an order for an available physical book, where purchasing is supported.
- The system shall calculate the total order amount.
- The system shall process payment through the configured payment gateway.
- The system shall generate an invoice after successful payment.
- Users shall be able to view their previous orders and order status.
- Users shall be able to request an unavailable book for future availability and purchase.

5.7 Payment Gateway

- The system shall integrate with an external payment gateway.
- The payment gateway shall securely process online transactions.
- The system shall receive and verify the payment status.
- Successful transactions shall be recorded.
- Failed or cancelled transactions shall be handled appropriately.
- Possible payment providers include Razorpay, Stripe, or another approved provider.

5.8 Invoice

- The system shall generate an invoice for successful purchases or payments.
- Users shall be able to view invoice details.
- Invoice details may include invoice number, user information, book or order details, amount, payment status, and transaction date.

5.9 User Profile

Users shall be able to view and update their profile information, including:

- Name

- Email
- Mobile number
- Address
- WhatsApp Number

5.10 History

The system shall maintain user activity history such as:

- Previous book orders
- Issued books
- Returned books
- E-books accessed or read
- Completed reading activity
- Browsing history.
- Payment and transaction history

5.11 Book Recommendation

The system can provide personalized book recommendations based on:

- Previously issued books
- Previously purchased books
- E-books read
- Categories frequently accessed
- Reading and browsing history
- User preferences
- Ratings or reviews

An AI or recommendation component may be used to provide these recommendations. For a basic implementation, a content-based recommendation approach can be used.

5.12 Notifications and Wishlist

- Users shall be able to save books to a Wishlist.
- The system shall send notifications through supported communication channels such as email and WhatsApp.
- Notifications may include book issue confirmations, request approval or rejection, due-date reminders, e-book expiry reminders, fine or payment reminders, order confirmations, payment confirmations, Wishlist updates, and available offers or discounts.

5.13 Due Date and Fine Management

- The system shall store book issue dates and due dates.
- The system shall identify overdue books.
- The system shall calculate applicable fines according to the configured library rules.
- The system shall notify users about overdue books and applicable fines.
- The system may send reminders before the due date.

5.14 Admin Functional Requirements

1. Book Management

Admin shall be able to:

- Add books.
- Edit book information.
- Delete/deactivate books.
- View book details.

- Search books.
- 2. Inventory Management
Admin can:
 - View available copies.
 - View issued copies.
 - Track returned books.
 - Update inventory.
 - Monitor book availability.
- 3. Issue Request Management
Admin can:
 - View pending requests.
 - Approve requests.
 - Reject requests.
 - Monitor issued books.
 - Monitor overdue books.
 - Fulfil user requests when books become available.
- 4. User Management
Admin can:
 - View registered users.
 - View user activity.
 - Manage user accounts where required.

5. Administration and Permission Management

Admin can:

- Create additional administrator accounts.
- Assign specific tasks to administrators.
- Assign limited or full permissions based on the administrator's role.
- Manage administrator access when required.

6. AI, Programming Languages, External APIs and Communication: -

6.1 AI Component: - AI is an optional component of the LMS and can primarily be used for the book recommendation feature. The recommendation system can analyse:

- User reading history
- Previously issued books
- Previously purchased books
- E-book activity
- Book categories
- Browsing history
- User preferences
- Ratings or reviews, where applicable

Based on this information, the system can recommend books that are relevant to the user's interests. For a basic implementation, a content-based recommendation system can be used.

6.2 Programming Languages: - Depending on the selected architecture, the system may use:

Component	Technology
Frontend	HTML, CSS, JavaScript
UI Framework	Bootstrap / React
Backend	Java / Python / Node.js
Database	MySQL
API	REST API
AI	Python / Scikit-learn (if used)

The final technology stack will be selected based on the implementation requirements, team expertise, deployment environment, and project scope.

6.3 External APIs: - The system may communicate with external services for additional functionality.

1. Payment API: - Used to securely process online payments. Possible providers include:

- Razorpay
- Stripe

2. Email API: - Used to send:

- Registration emails
- Issue or access notifications
- Due-date reminders
- Payment and order confirmations

3. WhatsApp API: - Used to send:

- Issue or access reminders
- Due-date notifications
- Order and payment notifications

4. Book/ISBN API (Optional): - An external book API can be used to retrieve:

- Book metadata
- Author information
- ISBN
- Cover images
- Categories

6.4 Communication: - Communication between the frontend and backend will primarily use HTTP/HTTPS through REST APIs. JSON will be used as the main data exchange format. The backend may also communicate with external APIs of payment, email, WhatsApp, and book information services.

7. Non-Functional Requirements: - It describe how well the system should perform and the quality, security, reliability, and scalability standards it should maintain.

7.1 Performance: -

- The system should respond to normal user requests within an acceptable time.
- Book searches and filtering should return results quickly.
- Database queries should be optimized for frequently used operations.
- The system should support multiple users accessing the application simultaneously.
- API response times should remain low under normal operating conditions.
- Frequently accessed data may be cached to improve response time.
- Long-running operations should not unnecessarily block normal user requests.

7.2 Security: -

- User passwords to be securely hashed and never stored as plain text.
- Authentication will be required for protected features.
- Role-based access control to be implemented for users and administrators.
- HTTPS will be used for secure communication.
- Sensitive payment information will not be stored directly unless required and handled securely.
- Input validation shall be performed on user-supplied data.
- The application should protect against common vulnerabilities such as SQL injection and cross-site scripting (XSS).
- Sessions and authentication tokens will be securely managed.
- Users will only be allowed to access data and operations for which they have permission.
- Administrative permissions shall be controlled according to assigned roles.

7.3 Software Quality: -

- Reliability: The system should operate consistently and handle failures gracefully without unnecessary data loss.
- Usability: The interface should be simple, clear, and easy to understand for both users and administrators.
- Maintainability: The application should use modular and well-structured code so that future changes and enhancements can be made easily.
- Availability: The system should remain available during expected operating hours with minimal downtime.
- Testability: The system should support unit testing, integration testing, and other appropriate testing activities.
- Compatibility: The web application should work correctly on commonly used web browsers and supported devices.

7.4 Scalability: - The system will be designed such that it can support increasing numbers of:

- Users
- Books
- Transactions
- Issue and access requests
- API requests

Possible scalability approaches include:

- Caching

- Load balancing
- Horizontal scaling of backend services
- Cloud deployment
- Containerization using Docker

The architecture will also allow future additions such as a mobile application, more advanced AI-based recommendations, support for multiple libraries or branches, and notification services.